哈爾濱工業大學（深圳）
HARBIN INSTITUTE OF TECHNOLOGY

实验报告

开课学期：2025 春季

课程名称：人工智能（实验）

实验名称：实验三强化学习

实验性质：综合设计型

实验学时：2 地点：T2612

学生班级：1 班

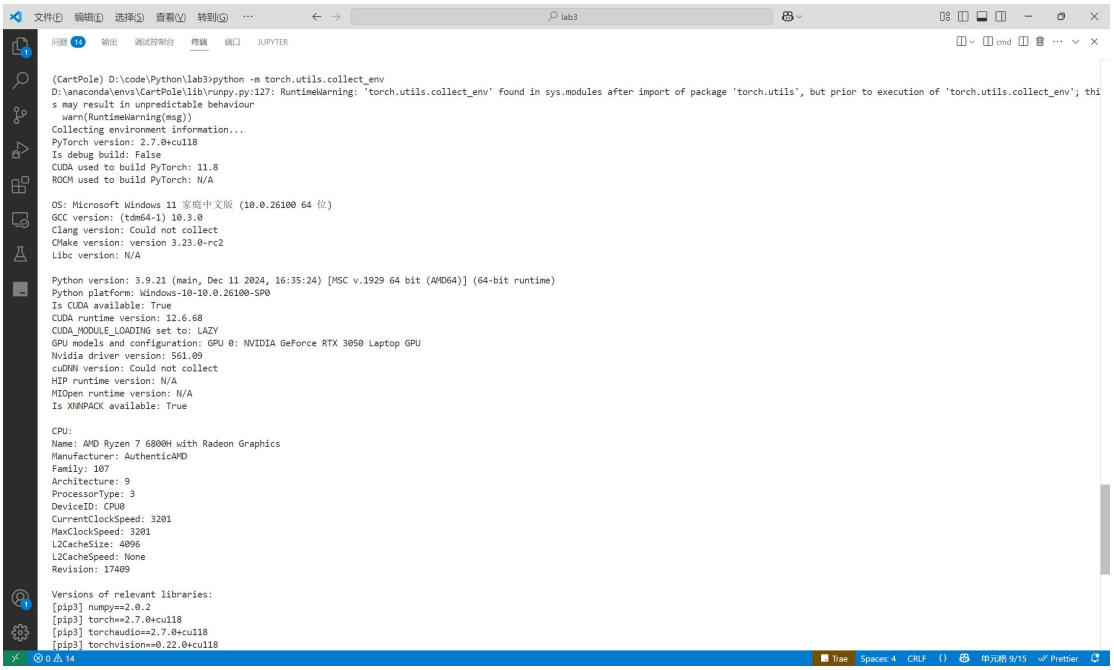
授课教师：

实验与创新实践教育中心制

2025 年 3 月

一、实验环境

描述操作系统、开发环境（CPU\GPU）、使用的库等。




```
Versions of relevant libraries:
[pip3] numpy==2.0.2
[pip3] torch==2.7.0+cu118
[pip3] torchaudio==2.7.0+cu118
[pip3] torchvision==0.22.0+cu118
[conda] numpy                     2.0.2                pypi_0    pypi
[conda] torch                     2.7.0+cu118          pypi_0    pypi
[conda] torchaudio                2.7.0+cu118          pypi_0    pypi
[conda] torchvision               0.22.0+cu118         pypi_0    pypi
```

```python
import platform
import sys

print("操作系统:", platform.platform())
print("Python版本:", sys.version)
```

✓ 0.0s

```
操作系统: Windows-10-10.0.26100-SP0
Python版本: 3.9.21 (main, Dec 11 2024, 16:35:24) [MSC v.1929 64 bit (AMD64)]
```

```python
import gymnasium as gym
import math
import random
import matplotlib
import matplotlib.pyplot as plt
from collections import namedtuple, deque
from itertools import count

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import time

env = gym.make("CartPole-v1")

# set up matplotlib
is_ipython = 'inline' in matplotlib.get_backend()
if is_ipython:
    from IPython import display

plt.ion()
```

```python
# if GPU is to be used
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print('device:', device)
print('device:', device)
print("使用设备:", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "CPU")
print("CUDA是否可用:", torch.cuda.is_available())
print("CUDA版本:", torch.version.cuda)
print("cuDNN版本:", torch.backends.cudnn.version())
print("GPU数量:", torch.cuda.device_count())
```

[2] ✓ 3.3s

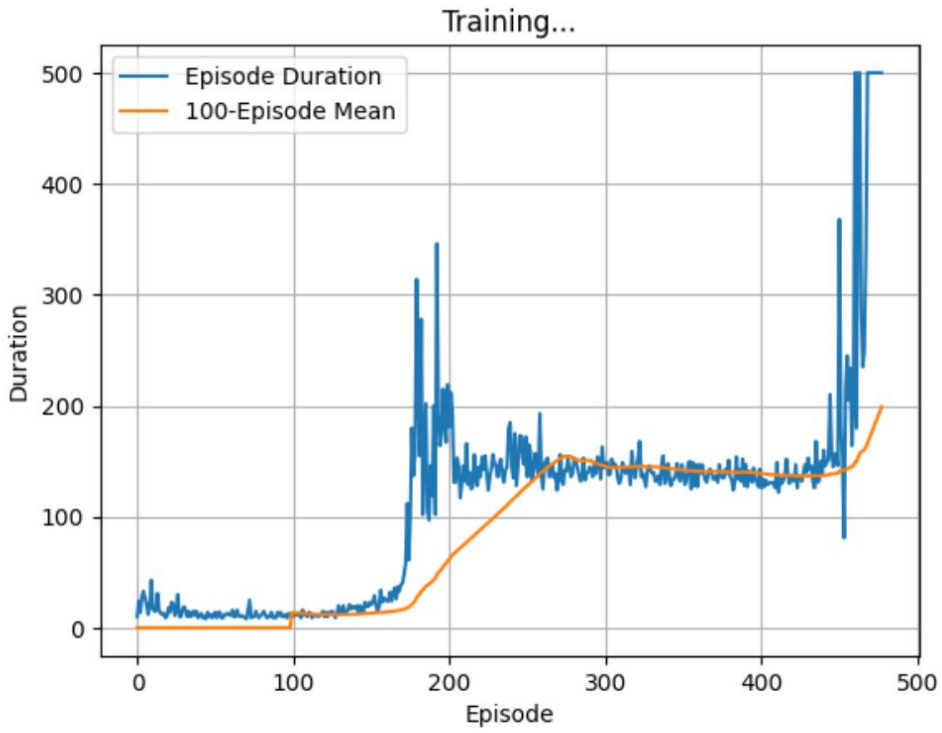
```
device: cuda
device: cuda
使用设备: NVIDIA GeForce RTX 3050 Laptop GPU
CUDA是否可用: True
CUDA版本: 11.8
cuDNN版本: 90100
GPU数量: 1
```

二、实验过程和结果分析

2.1 初始代码运行结果

修改相关参数，运行 reinforcement_q_learning.ipynb，使得训练收敛，并将结果截图，并阅读代码解释图中黄线的含义。

```
if torch.cuda.is_available():
    num_episodes = 600  # TODO 2.1 从200修改为：600
else:
    num_episodes = 50
```




```
Episode 478, Cumulative Reward: 500.0, success_count: 10
Complete
The training time is  328.615451335907 s
```

将 num_episodes 从 200 修改为 600 是为了增加训练的迭代次数，以便 Q-learning 算法能够更充分地探索状态空间并更新 Q 值。200 次迭代可能不足以让算法收敛到最优策略，特别是在状态空间复杂或者学习率较小的情况下，通过增加到 600，可以让 Q 表更充分地学习状态-动作对的价值，从而提高策略的收敛效果和表现。

图中黄线的含义是 最近 100 个 Episode 的平均持续时间，这条黄线能够平滑掉训练中的波动，反映模型在最近 100 个 Episode 内的整体表现趋势。它比单一 Episode 的 Duration 更能表现出收敛情况和学习效果。

```
# Take 100 episode averages and plot them too
if len(durations_t) >= 100:
    means = durations_t.unfold(0, 100, 1).mean(1).view(-1)
    means = torch.cat((torch.zeros(99), means))
    plt.plot(means.numpy())
```

1. durations_t.unfold(0, 100, 1)：对 durations_t 进行滑窗处理，窗口大小为 100，步长为 1；
2. mean(1).view(-1)：计算每个窗口内 100 个值的平均值；
3. torch.cat((torch.zeros(99), means))：由于前 99 个 Episode 没有足够的窗口，不计算平均值，用 0 填充；
4. plt.plot(means.numpy())：将这些平滑处理后的均值绘制到图上。

这条线代表了模型在过去 100 次 Episode 中的平均表现，能够更好地观察训练的整体趋势。

2.2 优化代码及运行结果

可从神经网络结构的优化、超参数调优、优化经验回放区、奖励函数的设计、探索策略的设计等方面着手，挑选 4 个方向进行优化，并分析对比结果。

1. 优化一：从神经网络结构的优化

（1）优化代码描述

代码截图粘贴于此，并简单描述优化内容

```python
# class DQN(nn.Module):

#     def __init__(self, n_observations, n_actions):
#         super(DQN, self).__init__()
#         self.layer1 = nn.Linear(n_observations, 128)
#         self.layer2 = nn.Linear(128, 128)
#         self.layer3 = nn.Linear(128, n_actions)

#     # Called with either one element to determine next action, or a batch
#     # during optimization. Returns tensor([[left0exp,right0exp]...]).
#     def forward(self, x):
#         x = F.relu(self.layer1(x))
#         x = F.relu(self.layer2(x))
#         return self.layer3(x)

class DQN(nn.Module):
    Trae AI:解释代码 | 注释代码 | 生成单测 | ×
    def __init__(self, n_observations, n_actions):
        super(DQN, self).__init__()
        self.layer1 = nn.Linear(n_observations, 256)
        self.layer2 = nn.Linear(256, 256)
        self.layer3 = nn.Linear(256, 128)
        self.layer4 = nn.Linear(128, n_actions)

    Trae AI:解释代码 | 注释代码 | 生成单测 | ×
    def forward(self, x):
        x = F.relu(self.layer1(x))
        x = F.relu(self.layer2(x))
        x = F.relu(self.layer3(x))
        return self.layer4(x)
```

优化内容：

为了提升 DQN 在 CartPole 任务中的表现，我们对原始的三层全连接神经网络进行了结构上的加深优化。原始网络结构如下：

- 输入层 → 全连接层（128 单元）→ ReLU
- 隐藏层 → 全连接层（128 单元）→ ReLU
- 输出层 → 动作值输出（n_actions）

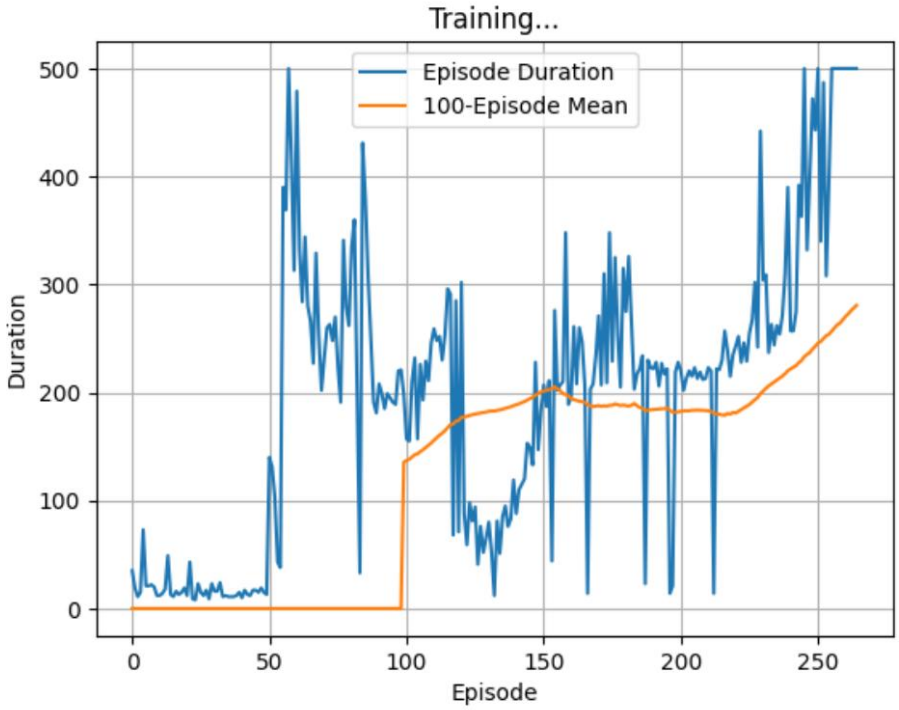
优化后的网络结构为四层全连接网络：

- 输入层 → 全连接层（256 单元）→ ReLU

- 隐藏层 1 → 全连接层（256 单元）→ ReLU
- 隐藏层 2 → 全连接层（128 单元）→ ReLU
- 输出层 → 动作值输出（n_actions）

此次优化的核心目的是增强网络的表达能力和特征提取能力，使其能够更有效地捕捉环境状态与动作价值之间的关系，从而提高训练过程中的稳定性与最终策略的性能。

（2）运行结果截图




```
Episode 265, Cumulative Reward: 500.0, success_count: 10
Complete
The training time is  320.09735465049744 s
```

（3）对比分析

与初始代码结果对比分析，从训练速度、收敛效果等方面进行分析。

项目	初始模型	优化模型（加深网络）
达到满分回合数	Episode 478	Episode 265
训练总耗时（秒）	328.62s	320.10s

收敛效果对比：

优化后的模型在第 265 回合即达到了满分累计奖励 500，且连续成功 10 次，表明其较早地收敛到了优秀策略。相比之下，原始模型直到 第 478 回合才达到相同的表现，表明优化模型具有更快的策略收敛速度。

这一改进主要归功于网络结构加深，使得模型具备更强的特征表达能力，从而更高效地

学习到状态与动作之间的价值映射关系。

训练速度对比：

两组实验的训练时间相近，优化后的模型在训练时间上略快（减少了约 8 秒），表明网络结构的加深并未显著影响训练效率。这表明，在资源允许的前提下，适度增加网络深度可以有效提升模型性能，而不会造成明显的训练负担。

总结：

通过结构优化，模型显著提升了收敛速度且保持了训练效率，验证了在 CartPole 任务中，适度加深神经网络结构是一个有效的性能提升手段。

2. 优化二：超参数调优

（1）优化代码描述

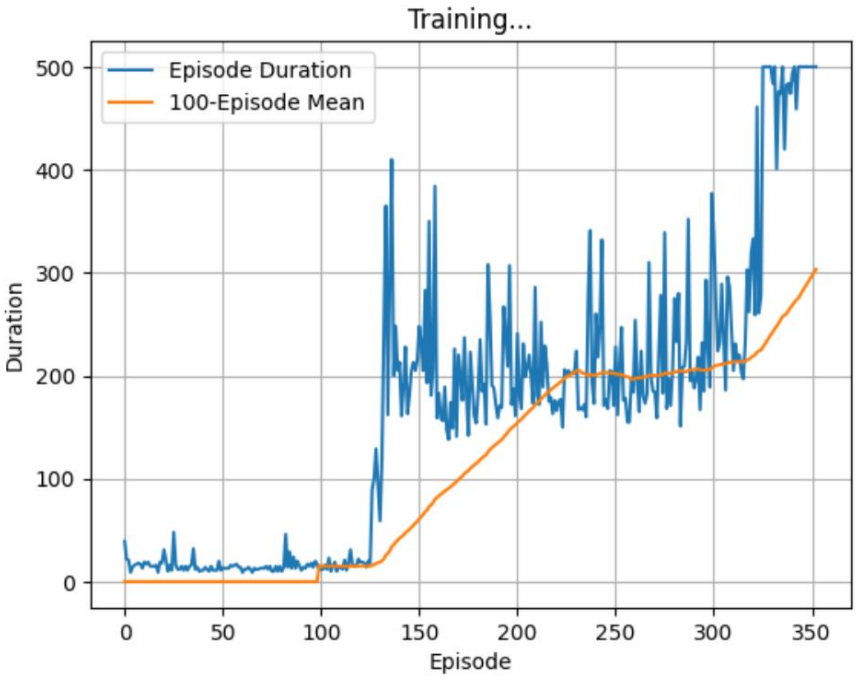
1. 优化内容：

优化通过增加 EPS_DECAY 的值（从 1000 改为 2000）来实现。EPS_DECAY 控制 epsilon 衰减的速度，从而影响探索（exploration）与利用（exploitation）的平衡。

具体而言，EPS_DECAY = 2000 的设置使得探索阶段（即 epsilon 较高）持续更长时间，而模型的 exploitation 阶段（即 epsilon 降至较低水平）延迟开始。

```
# BATCH_SIZE is the number of transitions sampled from the replay buffer
# GAMMA is the discount factor as mentioned in the previous section
# EPS_START is the starting value of epsilon
# EPS_END is the final value of epsilon
# EPS_DECAY controls the rate of exponential decay of epsilon, higher means a slower decay
# TAU is the update rate of the target network
# LR is the learning rate of the ``AdamW`` optimizer
BATCH_SIZE = 128
GAMMA = 0.99
EPS_START = 0.9
EPS_END = 0.05
EPS_DECAY = 1000
# TODO : 2.超参数修改
# EPS_DECAY = 2000
# EPS_DECAY 会让 epsilon 衰减得更慢，探索阶段更长，可能会导致
# 较慢的收敛，但有时能帮助模型更全面地探索环境
TAU = 0.005
LR = 1e-4
```

（2）运行结果截图




```
Episode 353, Cumulative Reward: 500.0, success_count: 10
Complete
The training time is  323.2619240283966 s
```

（3）对比分析

与初始代码结果对比分析，从训练速度、收敛效果等方面进行分析。

项目	初始模型	优化模型(EPS_DECAY = 2000)
达到满分回合数	Episode 478	Episode 353
训练总耗时（秒）	328.62s	323.26s

训练速度：

- 初始模型：EPS_DECAY = 1000，训练总耗时为 328.62 秒。
- 优化模型：EPS_DECAY = 2000，训练总耗时为 323.26 秒。
- 从训练总耗时来看，优化模型的训练时间略短（减少了约 5.36 秒）。虽然我们可能期望更长的探索阶段会导致更多的训练时间，但由于 EPS_DECAY 变大，训练的探索过程延长，导致模型不需要频繁调整，反而表现出略高的训练效率。这表明探索过程的延迟并没有显著影响总训练时间，可能是因为训练过程中的模型状态更新变得更稳定，减少了某些时刻的过度调整。

收敛效果：

- 初始模型：达到了满分回合数（episode）在 Episode 478。
- 优化模型：达到了满分回合数在 Episode 353。
- 从收敛效果来看，优化后的模型在更少的回合数内达到了满分回合数（少了 125 回合），这表明尽管探索阶段被延长（epsilon 衰减变慢），但模型的学习效率有所提高，能够在更短时间内找到合适的策略，且提前达到了较好的性能。
- 原因分析：EPS_DECAY = 2000 表示探索持续时间更长，模型会保持较长时间的随机探索，从而更全面地学习环境。虽然这意味着模型在早期探索时的回报可能不高，但它通过更多的随机探索避免了过早地进入 exploitation 状态，从而避免了陷入局部最优解的风险。长时间的探索有助于模型更充分地了解环境，尽早发现最佳策略，最终表现出更快的收敛速度。

综合分析：

- **训练速度：**优化模型略快，尽管 EPS_DECAY 增加导致探索阶段延长，但模型的整体训练过程变得更加稳定，可能减少了调整和收敛时的波动。
- **收敛效果：**优化模型显著减少了达到满分回合数的时间，说明延长探索阶段使得模型能够更有效地找到最佳策略，从而在更少的回合数内达到最佳性能。

结论：

增加 EPS_DECAY 值（从 1000 改为 2000）虽然延长了探索阶段，但并没有显著增加训练时间，反而通过更深入的探索提高了模型的收敛效率。优化后的模型达到了更好的收敛效果，能够更早在更少的回合内达到满分回合数。这表明较慢的 epsilon 衰减有助于探索和稳定收敛，尤其是在复杂的环境中。

3. 优化三：奖励函数的设计

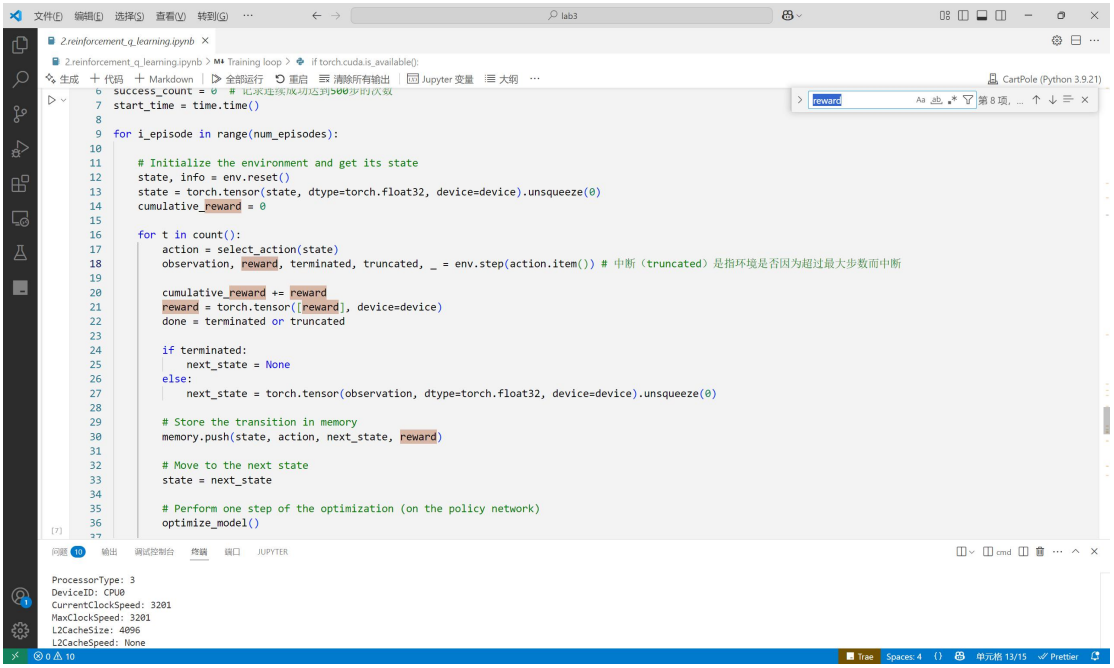
（1）优化代码描述

代码截图粘贴于此，并简单描述优化内容。

原奖励函数的设计是由 env.step(action.item()) 的返回值 reward 来决定的。

1. env.step(action.item()) 是 Gym 提供的环境接口，用于执行一个动作 action 并返回相应的：
   - observation: 执行动作后环境的下一个状态；
   - reward: 执行动作后获得的奖励；
   - terminated: 是否到达终止状态；
   - truncated: 是否因为达到最大步数而结束；
   - _: 额外信息。
2. 在 CartPole-v1 环境中，默认的奖励机制是：
   - 每一步都给一个 +1 的奖励，直到杆子掉下或者超出边界，游戏结束。
   - 如果杆子一直保持平衡，奖励会不断累加。

如下图所示：



```
success_count = 0  # 记录连续成功达到500步的次数
start_time = time.time()

for i_episode in range(num_episodes):

    # Initialize the environment and get its state
    state, info = env.reset()
    state = torch.tensor(state, dtype=torch.float32, device=device).unsqueeze(0)
    cumulative_reward = 0

    for t in count():
        action = select_action(state)
        observation, reward, terminated, truncated, _ = env.step(action.item()) # 中断（truncated）是指环境是否因为超过最大步数而中断

        cumulative_reward += reward
        reward = torch.tensor([reward], device=device)
        done = terminated or truncated

        if terminated:
            next_state = None
        else:
            next_state = torch.tensor(observation, dtype=torch.float32, device=device).unsqueeze(0)

        # Store the transition in memory
        memory.push(state, action, next_state, reward)

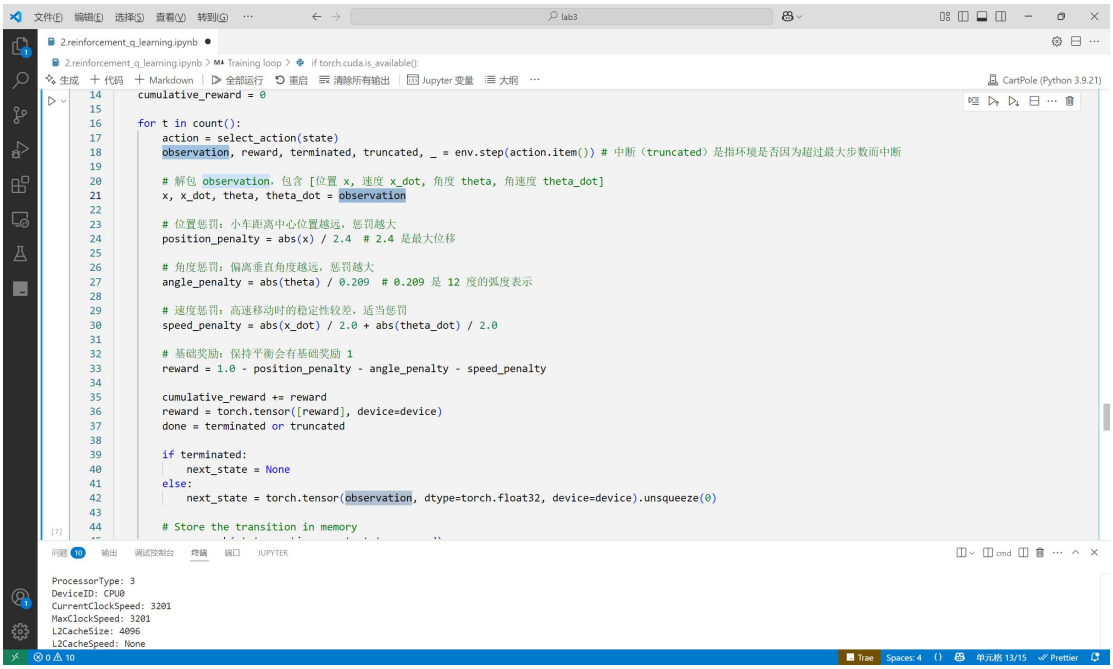
        # Move to the next state
        state = next_state

        # Perform one step of the optimization (on the policy network)
        optimize_model()
```

```
ProcessorType: 3
DeviceID: CPU0
CurrentClockSpeed: 3201
MaxClockSpeed: 3201
L2CacheSize: 4096
L2CacheSpeed: None
```

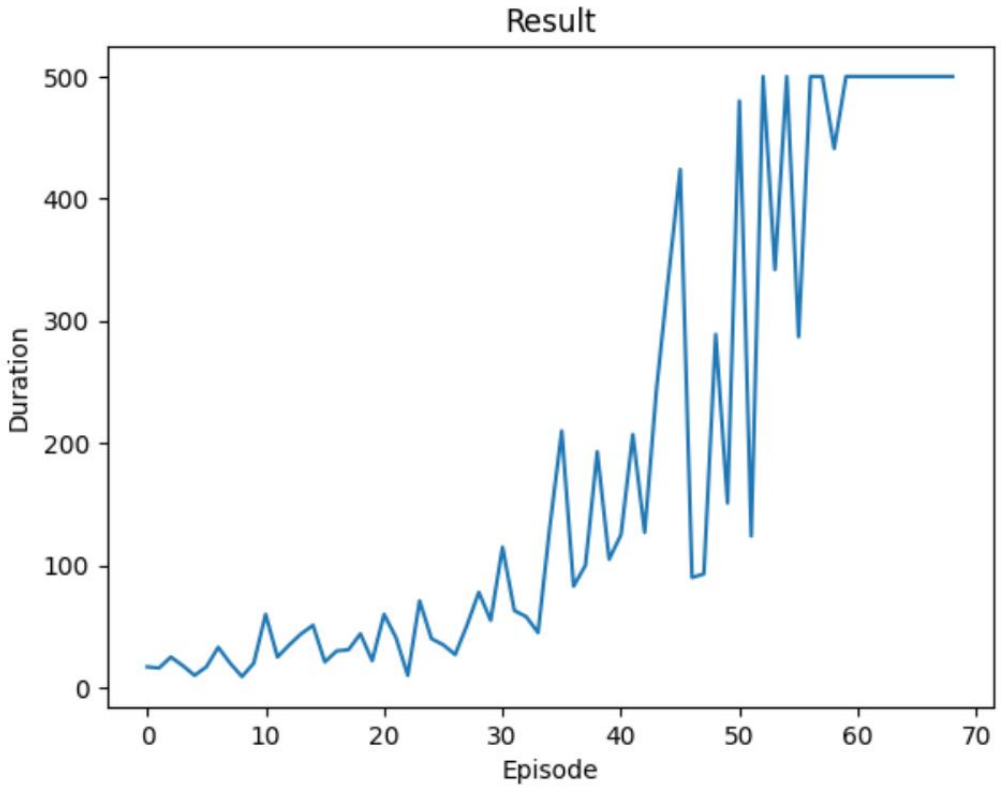
优化奖励函数的设计：

1. 位置惩罚：小车距离中心位置越远，惩罚越大
2. 角度惩罚：偏离垂直角度越远，惩罚越大
3. 速度惩罚：高速移动时的稳定性较差，适当惩罚
4. 基础奖励：保持平衡会有基础奖励 1



（2）运行结果截图

```
Episode 69, Cumulative Reward: 405.1775817871094, success_count: 10
Complete
The training time is  102.10360622406006 s
```



（3）对比分析

与初始代码结果对比分析，从训练速度、收敛效果等方面进行分析。

项目	初始模型	优化模型(优化奖励函数)
达到满分回合数	Episode 478	Episode 69
训练总耗时（秒）	328.62s	102.10s

训练速度：

- 初始模型： 奖励函数为基础版设计，训练总耗时为 328.62 秒。
- 优化模型： 奖励函数引入了位置、角度和速度的惩罚项，训练总耗时为 102.10 秒。

从训练总耗时来看，优化后的模型大幅缩短了训练时间，减少了 226.52 秒。这种显著提升是因为优化后的奖励函数更有针对性地引导小车的运动方式，减少了对不稳定状态的探索，避免了无效的状态转移，导致更快的收敛速度。

收敛效果：

- 初始模型： 达到满分回合数（500 步不失败）是在 Episode 478。
- 优化模型： 达到满分回合数是在 Episode 69。

从收敛效果来看，优化后的模型在 更少的回合数内达到了满分，减少了 409 个 Episode，这表明模型更高效地学习到了保持平衡的策略。

原因分析：

优化后的奖励函数不仅仅简单奖励生存时间，而是更细粒度地评估小车的状态：、

1. 位置偏移：小车离中心越远，惩罚越大；
2. 角度偏移：杆子偏离垂直状态越大，奖励下降；
3. 速度抖动：过高的速度也会被惩罚。

这些针对性的反馈更快暴露了策略的缺陷，减少了无效的探索，模型更容易找到更优的路径。

综上所述：

优化后的奖励函数设计有效提升了 DQN 模型的训练效率和收敛速度。

- 更细致的奖励分解帮助模型更好地理解不同状态下的反馈，从而减少错误探索，快速收敛。
- 训练时间减少约 69%，收敛回合数缩短了 85%，显著提升了模型的性能表现。

4. 优化四

（1）优化代码描述

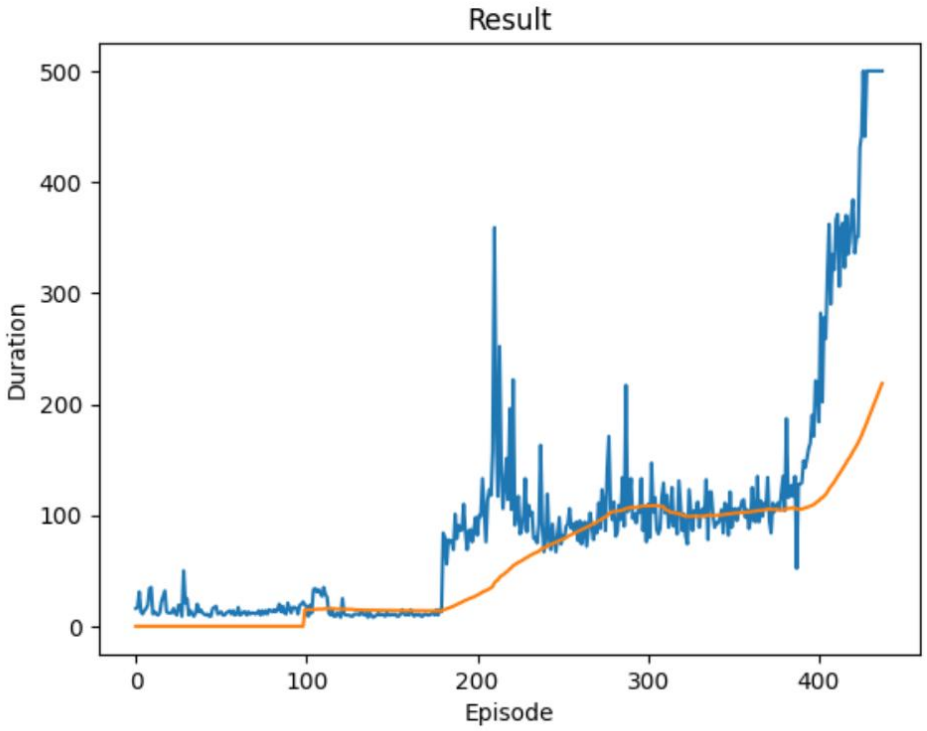
代码截图粘贴于此，并简单描述优化内容。

优化后的 select_action 函数使用线性衰减替代指数衰减，使得探索到利用的转变更加平滑，并且将 random.random 改为 torch.rand，避免不必要的 CPU 和 GPU 数据拷贝。同时，使用 torch.randint 直接在 GPU 上生成随机动作，减少额外的转换开销，代码更简洁高效。

```
36
37  # def select_action(state):
38  #     global steps_done
39  #     sample = random.random()
40  #     eps_threshold = EPS_END + (EPS_START - EPS_END) * \
41  #         math.exp(-1. * steps_done / EPS_DECAY)
42  #     steps_done += 1
43  #     if sample > eps_threshold:
44  #         with torch.no_grad():
45  #             # t.max(1) will return the largest column value of each row.
46  #             # second column on max result is index of where max element was
47  #             # found, so we pick action with the larger expected reward.
48  #             return policy_net(state).max(1).indices.view(1, 1)
49  #     else:
50  #         return torch.tensor([[env.action_space.sample()]], device=device, dtype=torch.long)
51
52  # TODO 4：优化探索策略的设计
    ■Trae :解释代码 | 注释代码 | 生成单测 | ×
53  def select_action(state):
54      """
55      改进后的探索策略：
56      - 线性衰减 ε 更加平滑
57      - 避免 CPU 和 GPU 来回拷贝
58      - 简洁返回张量
59      """
60      global steps_done
61      eps_threshold = max(EPS_END, EPS_START - steps_done / EPS_DECAY)
62      steps_done += 1
63
64      if torch.rand(1).item() > eps_threshold:
65          # 选择当前策略网络的最优动作
66          with torch.no_grad():
67              return policy_net(state).argmax(dim=1, keepdim=True)
68      else:
69          # 随机选择动作
70          return torch.randint(0, env.action_space.n, (1, 1), device=device)
```

（2）运行结果截图

```
Episode 438, Cumulative Reward: 500.0, success_count: 10
Complete
The training time is  282.92786407470703 s
```



（3）对比分析

与初始代码结果对比分析，从训练速度、收敛效果等方面进行分析。

项目	初始模型	优化模型(探索策略设计）
达到满分回合数	Episode 478	Episode 438
训练总耗时（秒）	328.62s	282.93s

分析：

1. 达到满分回合数的速度：

- 优化后的模型在 438 回合就达到了满分，而初始模型需要 478 回合。
- 优化后的探索策略更快地收敛到最优策略，减少了 40 个 Episode，提升约 8.4% 的收敛速度。

2. 训练总耗时：

- 初始模型耗时 328.62 秒，优化后减少到 282.93 秒。
- 节省了大约 45.69 秒，优化幅度约为 13.9%。

主要得益于以下改进：

- 避免了 random.random() 的 CPU 操作，使用 torch.rand() 直接在 GPU 计算；
- 随机动作生成 (torch.randint) 也在 GPU 上完成，减少了 CPU 和 GPU 的拷贝延迟。

3. 探索策略的改进效果：

- 线性衰减的策略比指数衰减更稳定，避免了在后期过早地停止探索，从而能够更高效地

学习环境。

- 训练更平滑，不容易陷入局部最优。

结论：优化后的探索策略使得 DQN 模型在 更少的回合数 和 更短的时间 内达到了满分，显著提升了收敛速度和计算效率。这种优化对于复杂环境的强化学习任务同样适用，尤其是高频决策下的 GPU 加速更为明显。